ML_Optimization_Problem3

2025-12-03

STSCI 6520 Homework 2 - Problem 3

K.CHAN

The problem is about building a boosting algorithm.

The Data: There are two classes, $+1$ and -1. - Class $+1$ is centered at $(1, 1)$. - Class -1 is centered at $(-1, -1)$.

They have the same variance, $2I$, so their areas overlap significantly. The Weak Learner (Part b):Usually, a linear classifier uses a weight vector $w = (w_1, w_2)$ and predicts based on $w_1x_1 + w_2x_2$.

The "weak" learner here is restricted to looking at only one coordinate at a time. - It's like asking: "Is x_1 positive?" or "Is x_2 positive?". - The formula $\hat{v} = \arg\max |\sum Y_i \langle X_i, v \rangle|$ simply picks the coordinate direction that correlates most strongly with the labels Y.

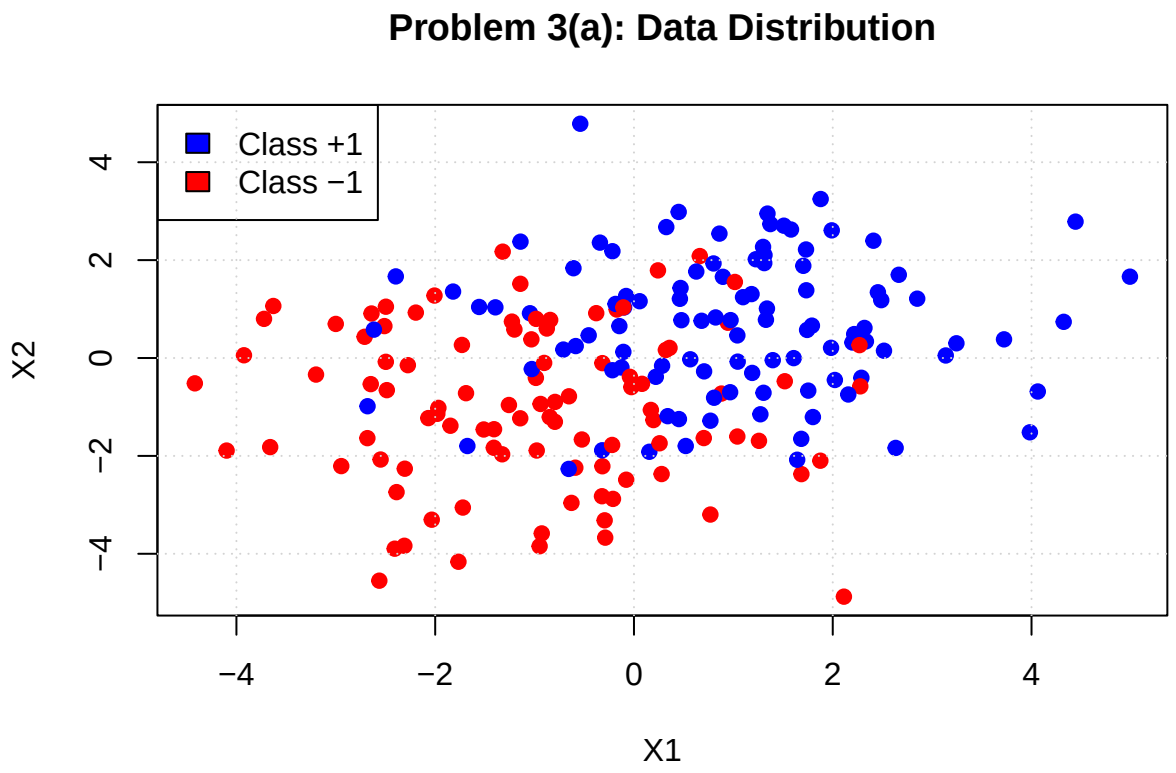
- The Boosting Loop (Part c):Instead of picking just one coordinate, we iteratively build a complex classifier θ.
- Weights (η): These track how "important" each data point is. - Hard-to-classify points get higher weights.Selection (v_{t+1}): - We pick the coordinate direction that best separates the weighted data.
- Step Size (α_t): This calculates how much we should trust this new direction.
- Update (θ): We add this new weighted direction to our total classifier.

```
## Boosting with Weak Linear Classifiers
set.seed(8888)
# 1. Data Gennnnn

# Function to generate data
# n: number of samples
# Returns a list containing X (matrix) and Y (vector)
generate_data <- function(n) {
  # Means
  mu_neg <- c(-1, -1)
  mu_pos <- c(1, 1)

  # Identity matrix scaled by 2
  Sigma <- diag(2, 2)

  Y <- sample(c(-1, 1), n, replace = TRUE)
  X <- matrix(0, nrow = n, ncol = 2)

  for(i in 1:n) {
    if(Y[i] == 1) {
```

```
      X[i, ] <- mu_pos + rnorm(2, mean=0, sd=sqrt(2))
    } else {
      X[i, ] <- mu_neg + rnorm(2, mean=0, sd=sqrt(2))
    }
  }
  return(list(X=X, Y=Y))
}

# (a) Generate and Plot n=200 points
train_data <- generate_data(200)
X_train <- train_data$X
Y_train <- train_data$Y
```

Final Solution 3A - PLOT

```
# Plotting
plot(X_train, col = ifelse(Y_train==1, "blue", "red"), pch=19,
     xlab="X1", ylab="X2", main="Problem 3(a): Data Distribution")
legend("topleft", legend=c("Class +1", "Class -1"), fill=c("blue", "red"))
grid()
```



2. 3B - Single Weak Classifier

```
# Coordinate vectors
e1 <- c(1, 0)
e2 <- c(0, 1)

# Function to train weak learner
# Returns the chosen vector v (either e1 or e2)
train_weak_learner <- function(X, Y) {
  # Calculate score for e1: sum(Y * x1)
  score1 <- sum(Y * (X %*% e1))
  # Calculate score for e2: sum(Y * x2)
  score2 <- sum(Y * (X %*% e2))

  if(abs(score1) >= abs(score2)) {
    # However can usually flip direction but the WEAK classifier has a limitation, so in this version w
    # to the formula: max | sum Y <X, v> |.
    return(e1)
  } else {
    return(e2)
  }
}
```

```
# Train on training set
v_hat <- train_weak_learner(X_train, Y_train)
#cat("Selected direction v_hat:", v_hat, "\n")

# Generate fresh test set
test_data <- generate_data(100)
X_test <- test_data$X
Y_test <- test_data$Y

# Predict
preds <- sign(X_test %*% v_hat)
# Handle sign(0) edge case if any (though unlikely with continuous data)
preds[preds == 0] <- 1

# Calculate Error
test_error_b <- mean(preds != Y_test)
```

B Final Solution

Part (b) Test Error: 0.22

3C: Implement boosting algorithm for weak learner

```
run_boosting <- function(X, Y, X_test, Y_test, T_iters=50) {
  n <- nrow(X)

  # Precompute Z = Y * X (elementwise scaling of rows)
```

```r
# Z[i, ] = Y[i] * X[i, ]
Z <- X * Y

# Initialize
eta <- rep(1/n, n) # Weights
theta <- c(0, 0)   # Parameter vector

# History to store errors
train_errs <- numeric(T_iters)
test_errs <- numeric(T_iters)

# Define basis vectors
basis_vecs <- list(c(1,0), c(0,1))

for(t in 1:T_iters) {

  # i. Update direction
  # v_{t+1} = argmax | <eta, Z v> |
  # We check v = e1 and v = e2
  val1 <- abs(sum(eta * (Z %*% basis_vecs[[1]])))
  val2 <- abs(sum(eta * (Z %*% basis_vecs[[2]])))

  if(val1 >= val2) {
    v_curr <- basis_vecs[[1]]
  } else {
    v_curr <- basis_vecs[[2]]
  }

  # ii. Adaptive stepsize
  # alpha_t = eta^T Z v_{t+1}
  # Note: This is the weighted margin along the chosen direction
  alpha_t <- sum(eta * (Z %*% v_curr))

  # iii. Parameter update
  theta <- theta + alpha_t * v_curr

  # iv. Weight update
  # eta[i] proportional to eta[i] * exp( -alpha * y_i * x_i^T v )
  # Note that y_i * x_i^T v is exactly the i-th element of (Z %*% v_curr)
  margins <- Z %*% v_curr
  exponent_term <- exp(-alpha_t * margins)

  eta <- eta * exponent_term

  # Normalize eta to prevent numerical underflow/overflow and keep it a distribution
  eta <- eta / sum(eta)

  # --- Evaluate Error for Plotting ---

  # Train Error
  train_preds <- sign(X %*% theta)
  train_preds[train_preds == 0] <- 1
  train_errs[t] <- mean(train_preds != Y)
```

```
    # Test Error
    test_preds <- sign(X_test %*% theta)
    test_preds[test_preds == 0] <- 1
    test_errs[t] <- mean(test_preds != Y_test)
  }

  return(list(train_err = train_errs, test_err = test_errs, theta = theta))
}
```

```
# Run Boosting
T_total <- 100
boost_res <- run_boosting(X_train, Y_train, X_test, Y_test, T_iters = T_total)

# Plotting Errors
# Setup data frame for plotting
plot_df <- data.frame(
  Iteration = rep(1:T_total, 2),
  Error = c(boost_res$train_err, boost_res$test_err),
  Type = rep(c("Train", "Test"), each = T_total)
)

# plot_df %>% pivot_wider(names_from = Type, values_from = Error) %>%
#   ggplot(aes(x = Iteration)) +
#   geom_line(aes(y = Train, color = "Train Error"), size = 1) +
#   geom_line(aes(y = Test, color = "Test Error"), size = 1) +
#   labs(title = "Boosting Training vs Test Error",
#        x = "Iteration T",
#        y = "Error Rate") +
#   scale_color_manual(values = c("Train Error" = "blue", "Test Error" = "red")) +
#   theme_minimal() +
#   theme(legend.title=element_blank())

#ERRORS for answer
#train<-  tail(boost_res$train_err, 1)
#test <-  tail(boost_res$test_err, 1)
#theta <- boost_res$theta

plot(1:T_total, boost_res$train_err, type="l", col="blue", lwd=2, ylim=c(0, 0.5),
     main="Boosting Training vs Test Error", xlab="Iteration T", ylab="Error Rate")
lines(1:T_total, boost_res$test_err, col="red", lwd=2)
legend("topright", legend=c("Train Error", "Test Error"), col=c("blue", "red"), lwd=2)
grid()
```

3c Final Solution

- Final Training Error: 0.76
- Final Test Error: 0.78
- Final Theta: -0.1662887